

课程笔记

CS224R Lecture 19: Q-Learning 复习与总结

Anakiette 与 Stanford CS224R 教学团队整理

2026 年 4 月 3 日



视频作者/频道: Stanford Online

发布日期: 2025

视频时长: 约 60 分钟

讲者: Anakiette (TA)

课程主页: cs224r.stanford.edu

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 MDP 基础回顾	3
1.1 马尔可夫决策过程	3
1.2 值函数与 Q 函数	3
1.3 Bellman 方程	3
1.4 策略迭代与价值迭代	4
1.5 本章小结	4
2 表格型 Q-Learning	4
2.1 Q-Learning 算法	4
2.2 探索策略与奖励缩放	5
2.3 收敛性条件	5
2.4 本章小结	5
3 Fitted Q-Iteration 与深度 Q 网络	5
3.1 从表格到函数近似	5
3.2 DQN 的关键技巧	6
3.3 DQN 的改进	6
3.4 本章小结	6
4 从 DQN 到 Actor-Critic	7
4.1 Q-Learning 的局限	7
4.2 Actor-Critic 方法	7
4.3 本章小结	7
5 Q-Learning 系统实践与监控	7
5.1 监控指标与数据质量	7
5.2 经验回放策略与数据重平衡	8
5.3 本章小结	8
6 深度 Q 网络变体与训练对比	8
6.1 关键变体剖析	8
6.2 训练与推理成本对比	9
6.3 本章小结	9
7 应用案例与演化时序	9
7.1 Benchmark 背景与时间线	9
7.2 案例洞察	10
7.3 本章小结	10
8 常见故障与治理	10
8.1 故障共性	10
8.2 干预机制	10
8.3 本章小结	11

9 术语表与工具	11
9.1 关键术语速查	11
9.2 本章小结	11
9.3 本章小结	11
10 评估策略与实验设计	11
10.1 离线指标与对比	11
10.2 在线对照与干预	12
10.3 本章小结	12
11 可解释性、调试与自动化	12
11.1 Q 值可视化与诊断	12
11.2 训练流水线的自动化	13
11.3 本章小结	13
12 未来方向与开放问题	13
12.1 长期依赖与任务组合	13
12.2 治理与可迁移性	13
12.3 本章小结	14
13 总结与延伸	14
13.1 总结表	14
13.2 进一步阅读	15
13.3 本章小结	15

1 MDP 基础回顾

本讲由助教 Anakietette 主讲，作为课程总复习，从 MDP 基础出发回顾 Q-learning 的完整理论链条。

“We’re going to start off by just doing a very very brief overview of MDPs.” 这句开场既是提示，也是学习节奏的指南，提醒我们在完整的理论链条中先建立形式化符号体系，再逐层深入细节。

1.1 马尔可夫决策过程

MDP 五元组

MDP 由 $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ 定义：

- $\mathcal{S}$: 状态空间
- $\mathcal{A}$: 动作空间
- $P(s'|s, a)$: 状态转移概率
- $R(s, a)$ 或 $R(s, a, s')$: 奖励函数
- $\gamma \in [0, 1)$: 折扣因子

Markov 性：下一个状态仅依赖当前状态和动作，与历史无关。

1.2 值函数与 Q 函数

核心定义

状态值函数：在状态 s 下遵循策略 π 的期望累积回报

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right]$$

动作值函数 (Q 函数)：在状态 s 下执行动作 a 后遵循策略 π 的期望累积回报

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a \right]$$

两者的关系： $V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$

1.3 Bellman 方程

Bellman 期望方程

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \sum_{a'} \pi(a'|s') Q^\pi(s', a')$$

Bellman 最优方程：

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a')$$

1.4 策略迭代与价值迭代

迭代视角的互补

策略迭代 (policy iteration) 和价值迭代 (value iteration) 可以看作 Bellman 最优方程的两种实现路径：前者交替优化策略与价值函数，后者直接迭代最优价值。两者在收敛性质上都依赖于 Bellman 迭代的收缩性，但前者在每轮内对策略更新的精度更高，后者每步计算更简单。

步骤	特征
策略迭代	明确构造行为策略（与旧策略做对齐）、值函数收敛后再改策略
价值迭代	直接迭代 $V(s) \leftarrow \max_a [R + \gamma V(s')]$ ，省去显式策略存储
两者的关系	价值迭代可看成策略迭代的一个特例（策略只在最后提取）

表 1: 在离散 MDP 中常见的迭代算子对比

1.5 本章小结

MDP 提供了 RL 的形式化框架。值函数和 Q 函数是评价策略好坏的核心工具，Bellman 方程描述了它们的递归关系。

2 表格型 Q-Learning

2.1 Q-Learning 算法

Q-Learning 更新规则

Q-Learning 是一种 off-policy、model-free 的算法，直接学习最优 Q 函数：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

其中：

- α : 学习率
- $r + \gamma \max_{a'} Q(s', a')$: TD target (时序差分目标)
- $r + \gamma \max_{a'} Q(s', a') - Q(s, a)$: TD error

Q-Learning 的 Off-Policy 特性

Q-Learning 是 off-policy 的，因为它使用 $\max_{a'}$ 来计算 TD target，无论行为策略（用来收集数据的策略）是什么。这意味着我们可以用任意策略（如 ϵ -greedy）来探索，同时学习最优策略。

2.2 探索策略与奖励缩放

常见探索策略

- ϵ -greedy: 大部分时间选最优动作, $1 - \epsilon$ 概率探索, 适合离散动作空间。
- Boltzmann/softmax: 通过温度控制随机性, 便于把握不同动作的相对价值。
- Upper Confidence Bound (UCB): 结合置信区间, 在样本不足的状态自动提高探索权重。

奖励缩放与非平衡奖励

Q-Learning 对奖励分布敏感, 尺度过大导致 TD error 爆炸, 过小又让梯度消失。常用做法包括归一化奖励、基准线减去平均 reward, 以及对稀疏奖励做形状工程 (dense reward shaping)。在实践中, 配套的 instrumentation 应记录各状态-动作的 TD error 和 reward 分布, 以便及时调整缩放策略。

稀疏奖励中的虚假收敛

当大量状态从未收到正奖励时, agent 可能会在负奖励区域收敛到局部最优策略。建议定期检查 replay buffer 中正负样本的比例, 并通过 hindsight experience replay 或多任务辅助奖励增加密度。

2.3 收敛性条件

Q-Learning 的收敛保证

在表格型设定下 (有限状态和动作空间), 如果满足以下条件, Q-Learning 保证收敛到 Q^* :

1. 每个状态-动作对被访问无穷多次
2. 学习率满足 Robbins-Monro 条件: $\sum_t \alpha_t = \infty, \sum_t \alpha_t^2 < \infty$

2.4 本章小结

表格型 Q-Learning 通过 TD 更新直接逼近最优 Q 函数, 具有理论收敛保证。

3 Fitted Q-Iteration 与深度 Q 网络

3.1 从表格到函数近似

当状态空间连续或很大时, 无法维护 Q 值表格。解决方案是用函数近似器 (如神经网络) 来参数化 Q 函数。

Fitted Q-Iteration (FQI)

FQI 将 Q-Learning 转化为监督学习问题：

1. 用当前 Q 网络计算 TD target: $y_i = r_i + \gamma \max_{a'} Q_{\theta^-}(s'_i, a')$
2. 将 (s_i, a_i) 作为输入, y_i 作为标签, 训练 Q 网络:

$$\min_{\theta} \sum_i (Q_{\theta}(s_i, a_i) - y_i)^2$$

3. 更新目标网络 $\theta^- \leftarrow \theta$
4. 重复

3.2 DQN 的关键技巧

DQN (Deep Q-Network) 的创新

DQN 引入了两个关键技巧使深度 Q-Learning 稳定工作：

1. **Experience Replay**: 将历史经验存储在 replay buffer 中, 训练时随机采样 mini-batch。这打破了数据的时间相关性, 提高了样本利用率。
2. **Target Network**: 使用一个周期性更新的目标网络 Q_{θ^-} 来计算 TD target。这减少了“移动的目标”问题带来的不稳定性。

函数近似 Q-Learning 的不稳定性

与表格型不同, 使用函数近似的 Q-Learning **不保证收敛**。Bellman 更新的收缩性质在函数近似下可能失效, 导致：

- Q 值发散 (overestimation bias 的累积)
- 训练不稳定 (oscillation 或 divergence)
- 对超参数敏感

这就是为什么需要 target network、experience replay 等稳定化技巧。

3.3 DQN 的改进

- **Double DQN**: 用当前网络选择动作, 目标网络评估值, 减少 overestimation
- **Dueling DQN**: 将 Q 函数分解为 $V(s) + A(s, a)$, 分别估计状态值和优势
- **Prioritized Experience Replay**: 按 TD error 大小优先采样, 关注更有学习价值的经验

3.4 本章小结

从表格 Q-Learning 到 DQN 的核心跨越是函数近似。Experience replay 和 target network 是确保训练稳定的关键技巧。

4 从 DQN 到 Actor-Critic

4.1 Q-Learning 的局限

Q-Learning 在连续动作空间的困难

Q-Learning 需要计算 $\max_{a'} Q(s', a')$ ，在离散动作空间中可以遍历所有动作。但在连续动作空间中，这个最大化操作变成了一个连续优化问题，计算代价很高。

4.2 Actor-Critic 方法

解决方案：同时学习策略（actor）和值函数（critic）：

- **Actor**: $\pi_{\theta}(a|s)$ ，参数化策略，输出动作
- **Critic**: $Q_{\phi}(s, a)$ ，评估 actor 选择的动作好坏
- Actor 用策略梯度更新，critic 用 TD 学习更新

4.3 本章小结

Actor-Critic 方法结合了策略梯度和值函数方法的优点，是处理连续动作空间的主流范式。

5 Q-Learning 系统实践与监控

5.1 监控指标与数据质量

关键观测指标

- Replay buffer 覆盖率：衡量最近 N 个转移是否覆盖常见状态-动作对。
- TD error 分布：监控 TD error 的标准差与最大值，可提前感知数值爆炸。
- Action distribution drift：行为策略的采样频率是否偏向少数动作，防止训练走向狭窄策略。

为了做到稳定部署，工程团队必须把上述指标暴露到训练仪表盘，并结合基准数据制定警戒线。摄取的经验应被打标签（如挑战性、稀疏 reward、环境变化）以便后续分析。

Replay Buffer 覆盖与分布警示

如果 buffer 恰好被早期成功经验撑满，后续探索的样本无法进入训练，这会导致 agent 过早收敛。推荐设置 `min_size` 和 `max_size` 并定期用采样接口检查时间戳，必要时使用 `prioritized replay` 或 `rank-based` 重新激励采样。

5.2 经验回放策略与数据重平衡

Experience Replay 重平衡技巧

- Prioritized Replay: 以 $|\delta|^\alpha$ 作为采样权重, δ 为 TD error, α 控制偏好强度。
- Importance Sampling Correction: 对优先样本做比例补偿, 防止 policy bias。
- Diversity Buffer: 保留来自不同子任务或者模拟 seed 的经验片段, 避免训练陷入 narrow distribution。

直接将 replay buffer 当作一个黑匣子是不够的, 建议在每次训练迭代后记录新旧样本进入比例及其平均 reward。若新样本比例低于 30%, 说明探索效率下降, 需要加大学习率或 epsilon。

Replay buffer 过度依赖旧数据

随着训练推进, 旧数据可能已经过时, 尤其在非静态环境中。继续训练时要关注 buffer 中平均 timestep, 若超过 50% 的数据来自过去十分钟, 可启用循环清洗或将优先级重新置为高值, 避免旧策略主导梯度。

5.3 本章小结

本章强调 Q-Learning 系统建设中的可观测性与 replay buffer 的健康度, 任何一次大规模复训都应在数据收集、采样分布与 TD error 这三个维度设定 guardrail。

6 深度 Q 网络变体与训练对比

6.1 关键变体剖析

变体	核心机制	影响
Double DQN	当前网络选动作, 目标网络评估	抑制 overestimation, 稳定性提升
Dueling DQN	分解 $Q = V + A$	更快学到状态价值, 动作优势
Prioritized Replay	根据 TD error 加权采样	快速拾取稀有 but 关键的转移
Rainbow	多个增强技巧组合 (NoisyNet、Multi-step、Distributional)	覆盖性更广, 但调参更复杂

表 2: DQN 变体与其工程影响

多维组合的适用场景

不是所有场景都需要 Rainbow 的全套增强。在低容量环境下, Double DQN 结合 Dueling 已足以抑制过估。在富有计算资源的 Atari 任务或模拟驾驶中, Rainbow 多步回报与 NoisyNet 提供更强的泛化能力。

6.2 训练与推理成本对比

训练 vs 推理成本模型

训练集中于采样与优化： $Cost_{train} \approx N_{steps} \times (C_{forward} + C_{backward})$ ；推理则在吞吐与延迟间权衡： $Cost_{infer} \approx C_{forward} + C_{action\ selection}$ 。若将 DQN 推向嵌入式设备，应考虑量化 / 剪枝以减少计算量。

推理栈拆解

Inference pipeline 通常包括：环境感知 (state encoding)、Q 网络前向、arg max 动作选择、行动滤波 (如先验约束)、以及与模拟器接口的安全检查。在多 agent 协同中，还需把握通信与同步延迟。

低延迟推理中的过度量化

为了优化推理 latency，工程团队往往会对网络进行量化或剪枝，但如果没有保留关键路径 (如最后一层 advantage 分支)，Q 值排序可能被破坏。推荐保持双通道 (量化用于推理，full precision 备份用于 periodic calibration)。

6.3 本章小结

本章通过 DQN 变体对比与成本模型讨论，指出在不同资源约束下如何选择合适的增强技巧与算子组合。

7 应用案例与演化时序

7.1 Benchmark 背景与时间线

Q-Learning 演化时间线

- 1992 年：Watkins 的 Q-Learning 理论收敛。
- 2013 年：Mnih 等在 Atari 上用 DQN 实现 end-to-end 控制。
- 2015-2017 年：Double DQN、Dueling、Prioritized Replay、Rainbow 等陆续完善。
- 2016 年后：DQN 拓展到 MuJoCo、Roboschool 等连续控制任务，多 agent RL 与对抗训练开始流行。

提出 benchmark 时总会面对一个抉择：是用简单环境获取重复性、还是用复杂环境逼近真实应用。Stanford 的课程复习强调了前者的控制性与后者的代表性之间的 trade-off。

7.2 案例洞察

Atari 案例复盘

在 Atari Breakout 任务上，Agent 通过 frame stacking 提取动量信息，而 frame-skip 提高了训练效率。改变 reward shaping（例如加大 brick 击中奖励）后，agent 的策略发生了明显偏移，说明 reward scales 的调整容易触发新策略。

现实应用中的观测偏差

在机器人任务中，真实 sensor 噪声和模拟 sensor 差异可能导致 sim-to-real 的失败。即便在仿真中训练到高分，也必须做 domain randomization 并在部署前进行实际数据校对。

7.3 本章小结

本章结合 benchmark 发展与经典案例，提醒我们即便掌握了算法细节，也要当心仿真与真实世界之间的信息缺口。

8 常见故障与治理

8.1 故障共性

训练中的泛化失败

即使在训练环境中获得高 reward，agent 仍可能在稍有改动的初始状态下崩溃。原因通常是 replay buffer 过度依赖一小批高 reward 转移，难以覆盖状态空间。可以借助 entropy bonus 或多任务 replay 来缓解。

治理关注点

- Policy drift：需保留 dev 策略样本作为窗内基准。
- Reward hacking：强化学习中 reward shaping 要谨慎，不可通过简单修改 reward 让 agent 找到 shortcut。
- 数据透明：记录每次更新使用的数据 slice，并定期与业务方核对行为日志。

8.2 干预机制

Safe Interruptibility 与人为干预

在训练过程中，保持 interruptibility 意味着当人类检测到异常时，可以安全终止或重置 agent，而不会引发奖励爆炸。实践中会设置 kill switch，同时记录最后一次 TD error 以帮助定位原因。

过度干预带来的训练碎片化

频繁中断训练会导致 replay buffer 中数据来自不同策略，破坏 off-policy 假设。建议在干预后执行 warm-start offline training，确保新策略有充分时间收敛。

8.3 本章小结

治理层面需平衡自动训练与人为监控，干预机制应以 guardrail 为核心，避免重复训练碎片化。

9 术语表与工具

9.1 关键术语速查

术语	释义
TD error	$r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ ，用于衡量当前 Q 估计的更新量
Prioritized Experience Replay	根据 TD error 采样，并引入 IS correction
Target Network	周期性同步的网络，稳定 TD target
State Action Coverage	replay buffer 中不同 state-action 对的分布

表 3: 本讲涉及的重点术语

工具链盘点

常用工具包括：OpenAI Baselines（DQN 基础模板）、RLlib（支持多 agent）、MLFlow + TensorBoard（指标记录）、WeTest 或 Supersonic（在 edge 端量化验证）。

9.2 本章小结

将术语表与工具链一并掌握有助于从理论跨越到工程实践，尤其是在复现 benchmark 和上线模型时省去重复摸索。

9.3 本章小结

将术语表与工具链一并掌握有助于从理论跨越到工程实践，尤其是在复现 benchmark 和上线模型时省去重复摸索。

10 评估策略与实验设计

10.1 离线指标与对比

离线评估通常在与训练集不同的 holdout 环境中完成，以确保策略并未过拟合。常见指标包括累积 reward、episode 长度、稳定性（标准差）和 sample efficiency。下面的表格给出了这几个指标的关注点：

指标	备注
累计 reward	反映策略在固定起点下的任务完成度
Episode length	评估是否学会了更简洁的路径 / 效率
Sample efficiency	每单位环境交互带来的 reward 改善幅度
稳定性	不同 seeds 下 reward 的波动，过大会导致部署风险

表 4: 离线评估常用指标

Sample efficiency 与 reward ceiling 的权衡

有时候提升累计 reward 的边际收益非常低，但 sample efficiency 仍然未稳定，说明 agent 还在反复探索。建议在迭代曲线上同时跟踪两者，并在 plateau 期引入更强的 exploration 或 auxiliary task。

10.2 在线对照与干预

在生产环境下，可采用 champion/challenger 或 A/B 形式对照测试新策略。应保证 challenger 只在较小流量下运行，并准备回滚策略。

A/B vs Champion/ Challenger

A/B: 线上流量按照比例直接路由到 challenger，适合 behavior policy 与 baseline 差异较小时；Champion/Challenger: 在 shadow 环境运行 challenger，通过 offline evaluation 决策是否 promote。

评价分布偏移

训练环境与部署环境一旦存在观测噪声差异，离线指标的信号就可能失真。在安全敏感系统（如自动驾驶）中要优先建立 scenario 库用于 stress test。

10.3 本章小结

本章梳理了 Q-Learning 的评估体系，强调同时关注累积 reward、样本效率与稳定性，并建议在部署前设置对照测试与 stress test。

11 可解释性、调试与自动化

11.1 Q 值可视化与诊断

在训练过程中，可以通过 heatmap 或 contour 观察 Q 值随动作、状态变化的分布。若某个动作一直得到极高 Q 分数却从未选出，可能表明行为策略实现（如 ϵ -greedy）被 bug 影响。

诊断工具要素

重要信息包括：Q 值梯度、TD error 曲线、Replay buffer 中 action frequency。可用 TensorBoard、MLFlow 来展示时间序列图。

11.2 训练流水线的自动化

推荐在训练周期中嵌入自动化检查：每隔若干 epoch 进行 offline evaluation、同步 metrics 到 dashboard、自动计算 sample efficiency。训练 pipeline 还应支持 ‘checkpoint+resume’ 和 ‘metric gating’，后者在关键指标下降时自动暂停训练。

Replay buffer 哨兵

用一个轻量级模型实时计算 buffer 的 coverage 和 reward statistics，一旦发现分布尖锐偏移就发出警报。这种哨兵可以避免在早期就采集到 one-hot 数据导致后续训练崩溃。

自动化过度依赖历史指标

如果 gating 只看历史 reward 平均值，则可能忽略 recent decay。建议引入 sliding window version 的指标，将异常提高到 operator 手动确认。

11.3 本章小结

可解释性与自动化并行是规模化训练的基础，整理好诊断视图和自动 gating 机制能显著减少重启次数。

12 未来方向与开放问题

12.1 长期依赖与任务组合

Q-Learning 原始形式在处理长 horizon/ sparse reward 时会变得低效。现代方法倾向于多步 Q 更新 (multi-step TD)、hierarchical policy 以及外部 planner 的组合。

任务组合的 pipeline

通过把单一任务拆成 Macro / Micro 两层（如 meta policy 负责子任务选择，micro policy 负责具体控制）可以缓解 long-term credit assignment 的困难。

12.2 治理与可迁移性

开放问题清单

- Sim-to-real gap: 如何让 replay buffer 中真实数据发挥更大作用。
- Reward hacking: 长期 reward shaping 可能引入 shortcut，需定期复查 reward formula。
- 多模态状态: 当 robot state 包含图像、LiDAR 与语言，Q network 的处理 pipeline 复杂度骤增。

部署时的 drift 管理

即便策略在 offline 模拟中表现优异, Observation drift 或 action constraint drift 也会导致线上失效。建议配置周期性的 sanity check (如 evaluate on replayed expert rollouts)。

12.3 本章小结

本章指出 Q-Learning 领域仍在演进, 长期依赖、任务组合与治理审计是未来工作的主要方向。

13 总结与延伸

13.1 总结表

章节	关注点	后续建议
MDP 基础回顾	Bellman 迭代与策略/value 迭代的关系	复核收缩定理与演化形式
表格型 Q-Learning	Off-policy 收敛与探索均衡	为现有任务梳理 replay buffer 的覆盖率
Fitted Q-Iteration 与 DQN	函数逼近、稳定技巧与优化细节	在老系统中验证 target network / double DQN 影响
从 DQN 到 Actor-Critic	Continuous control 与 actor-critic 架构过渡	比较 deterministic policy gradient 与 Q-learning 的梯度估计
系统实践与监控	指标、replay buffer 健康度、干预机制	建立 instrumentation revoke 机制
变体与成本	Rainbow、成本模型、推理栈	规划训练/推理预算, 并保留挂回路
应用案例与治理	benchmark 与 sim-to-real 突破、治理关注点	规划 soft/hard interrupt 与 anomaly 回滚

表 5: 本讲重点内容与工程切入点

13.2 进一步阅读

延伸资料

- Watkins & Dayan, “Q-Learning,” Machine Learning 1992
- Mnih et al., “Human-level Control through Deep Reinforcement Learning (DQN),” Nature 2015
- Van Hasselt et al., “Deep Reinforcement Learning with Double Q-Learning,” AAAI 2016
- Lillicrap et al., “Continuous Control with Deep Reinforcement Learning (DDPG),” ICLR 2016
- Sutton & Barto, “Reinforcement Learning: An Introduction,” 2nd Edition, 2018
- OpenAI Safety Gym 2.0 文档（可做 interruptibility 测试）

13.3 本章小结

本讲以 MDP 与 Bellman 方程为基础，层层展开 Q-Learning 从表格到深度网络、再到 Actor-Critic 的演进，同时穿插系统实践、变体 COST 以及治理/应用案例，提供了完整的复习与操作 checklist。